SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. *(BL3)*

Assessment Tool Weightage: 10%

QUESTIONS: **Total Marks:50**

Annexure A

SCENARIO BASED TASK

Code of Conduct:

I PerarasanV (192521404) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Perarasan V

Annexure A

SCENARIO BASED TASK

QUESTIONS:

1 Scenario: Role of Parser vs Lexer

A student is developing a compiler for a simple calculator language where the lexical analyzer produces tokens such as NUM, PLUS, MINUS, STAR, DIVIDE, LPAREN, and RPAREN from the input expression. Although the lexer can identify and categorize symbols correctly, it does not ensure that the sequence of tokens forms a valid mathematical expression. The parser is responsible for analyzing this sequence based on grammar rules and constructing a syntax tree to represent the structure of the expression. In this scenario, the student must clearly differentiate between the roles of the lexer and parser and explain how the parser ensures syntactic correctness, handles operator precedence, and detects errors using a suitable example.

Role of Parser vs Lexer

In a compiler for a calculator language, the **lexer (lexical analyzer)** and **parser (syntax analyzer)** perform different but complementary tasks.

1. Role of the Lexer

The lexer reads the input character by character and converts it into a sequence of **tokens**. It identifies individual elements such as:

- NUM → numbers
- PLUS → +
- MINUS → -
- STAR → *
- DIVIDE → /
- LPAREN → (
- RPAREN →)

For example, the expression:

3 + 5 * 2

may be converted into:

NUM PLUS NUM STAR NUM

The lexer checks whether individual characters or words are valid tokens, but it **does not normally determine whether the entire sequence follows the grammar of a mathematical expression**.

2. Role of the Parser

The parser takes the token sequence produced by the lexer and checks it against the **grammar rules** of the calculator language.

A suitable grammar is:

```
expr   → term ((PLUS | MINUS) term)*
term   → factor ((STAR | DIVIDE) factor)*
factor → NUM | LPAREN expr RPAREN
```

This grammar also handles **operator precedence**:

- * and / have higher precedence than + and -.
- Parentheses have the highest priority.

For example, for:

```
3 + 5 * 2
```

the parser does not interpret it as (3 + 5) * 2. Instead, it constructs a syntax tree equivalent to:

```
   +
  / \
 3   *
    / \
   5   2
```

Thus, multiplication is performed before addition.

3. Detecting Syntax Errors

The parser also detects invalid token sequences. For example:

```
3 + * 5
```

produces tokens:

```
NUM PLUS STAR NUM
```

The lexer can correctly recognize every symbol, but the sequence is syntactically invalid because after PLUS, the grammar requires a term, which must begin with a factor. A STAR cannot occur there.

Similarly:

```
(3 + 5
```

is invalid because the opening parenthesis has no matching RPAREN. The parser reports a syntax error rather than constructing a valid syntax tree.

Conclusion

The lexer answers “What are these symbols?”, while the parser answers “Do these tokens form a valid expression, and how are they structured?” The lexer produces tokens, whereas the parser uses grammar rules to ensure syntactic correctness, enforce operator precedence and associativity, construct a parse tree/syntax tree, and report syntax errors.

2 Scenario: Left Recursion Problem

A compiler developer is implementing a recursive descent parser and observes that the grammar S → Sa | b causes the parser to enter an infinite loop. This issue arises because the grammar contains immediate left recursion, which is not suitable for top-down parsing techniques. Since the parser repeatedly expands the same non-terminal without consuming any input, it leads to non-termination. In this scenario, the developer must identify the problem of left recursion and demonstrate how to systematically eliminate it by rewriting the grammar into a form that supports predictive parsing.

Left Recursion Problem

The grammar is:

S → Sa | b

1. Identifying the Problem

This grammar contains **immediate left recursion** because the non-terminal S appears as the **first symbol on the right-hand side** of its own production:

S → S a

In a recursive descent parser, attempting to parse S results in:

S
→ S a
→ S a a
→ S a a a
→ ...

The parser keeps calling the procedure for S without consuming any input. Therefore, it enters an **infinite recursion/loop** and never terminates.

2. General Method for Eliminating Left Recursion

An immediate left-recursive grammar has the general form:

A → Aα | β

It can be transformed into:

A → βA'
A' → αA' | ε

where ε represents the empty string.

3. Applying the Method

Given:

S → Sa | b

Compare it with:

$A \rightarrow A\alpha \mid \beta$

Here:

- $A = S$
- $\alpha = a$
- $\beta = b$

Therefore, the left recursion can be eliminated as:

$S \rightarrow bS'$
$S' \rightarrow aS' \mid \varepsilon$

4. How the New Grammar Works

The new grammar generates strings such as:

b
ba
baa
baaa
...

For example, to generate baa:

S
→ bS'
→ baS'
→ baaS'
→ baa

The parser now consumes the input before recursively calling S', so it does not repeatedly expand the same non-terminal without making progress.

Conclusion

The original grammar $S \rightarrow Sa \mid b$ is **immediately left recursive** and is unsuitable for recursive descent or predictive parsing because it causes infinite recursion. By applying the standard left-recursion elimination technique, it becomes:

$S \rightarrow bS'$
$S' \rightarrow aS' \mid \varepsilon$

This equivalent grammar is suitable for **top-down predictive parsing** and allows the parser to terminate correctly.

3 Scenario: Dangling Else Problem

A team is designing a predictive parser for the grammar S → if E then S | if E then S else S | a. While constructing the LL(1) parsing table, they encounter conflicts due to ambiguity in associating the else clause with the correct if statement. This situation is known as the dangling else problem. To analyze the issue, the team must compute the FIRST and FOLLOW sets of the non-terminals and explain how overlapping entries in the parsing table arise. The scenario requires identifying the source of ambiguity and explaining why the grammar is not suitable for LL(1) parsing.

Dangling Else Problem

The given grammar is:

S → if E then S
| if E then S else S
| a

The **dangling else problem** occurs because an else can potentially be associated with more than one if. This creates ambiguity and causes a conflict when constructing an LL(1) parsing table.

1. FIRST Sets

Assuming E represents a valid expression and does not derive ε:

FIRST(S) = { if, a }

For the two alternatives beginning with if:

FIRST(if E then S) = { if }
FIRST(if E then S else S)= { if }
FIRST(a) = { a }

The important observation is that the first two productions have the **same FIRST set**:

{ if }

2. FOLLOW Set

Assuming S is the start symbol:

- Since S is the start symbol, $ FOLLOW(S).
- In if E then S, S occurs at the end, so FOLLOW(S) is propagated.
- In if E then S else S, the first S is followed by else, so else FOLLOW(S).
- The final S again receives FOLLOW(S).

Therefore:

FOLLOW(S) = { else, $ }

3. LL(1) Parsing Table Conflict

Consider the parsing-table entry for non-terminal S and lookahead if.

Both productions have if in their FIRST sets:

S → if E then S
S → if E then S else S

Therefore, the table entry:

M[S, if]

would have to contain **two different productions**:

M[S, if] = S → if E then S
S → if E then S else S

This is an **LL(1) conflict** because an LL(1) parser must be able to select exactly one production using only one lookahead token.

4. Source of the Dangling Else Ambiguity

Consider:

```
if E1 then
   if E2 then
      a
   else
      a
```

The else could be associated with the **inner** if or, theoretically, with the **outer** if.

The usual language convention is:

> An else is associated with the nearest unmatched if.

However, the given grammar does not explicitly enforce this rule, so it is **ambiguous**.

5. Why the Grammar Is Not LL(1)

The grammar is not suitable for LL(1) predictive parsing because:

1. The first two productions have overlapping FIRST sets:

FIRST(if E then S) = {if}
FIRST(if E then S else S) = {if}

2. Consequently, the parsing-table entry M[S, if] contains multiple productions.
3. One lookahead token (if) is insufficient to decide which production should be selected.
4. The grammar is ambiguous because the else can have different possible associations.

Conclusion

The dangling else problem demonstrates why the grammar

S → if E then S
| if E then S else S
| a

Can not directly be used to construct an **LL(1) predictive parser**. The overlapping FIRST sets create a parsing-table conflict, while the optional else creates the underlying ambiguity. A grammar transformation or an explicit rule such as **"else matches the nearest unmatched if"** is required to remove the ambiguity and

make predictive parsing possible.

4 Scenario: Shift-Reduce Parsing

A shift-reduce parser is used to process the input string id + id using the grammar E → E + T | T and T → id. The parser follows a bottom-up approach by maintaining a stack and performing shift and reduce operations to gradually derive the start symbol. At each step, the parser decides whether to shift the next input symbol onto the stack or reduce a sequence of symbols using a production rule. In this scenario, the parsing process must be traced step by step in a tabular format, showing the stack contents, remaining input, and the action taken at each stage.

Shift-Reduce Parsing

Given the grammar:

E → E + T
E → T
T → id

Input string:

id + id $

Here, $ indicates the end of input. A shift-reduce parser uses a **stack** and performs two main operations:

- **Shift:** Move the next input symbol onto the stack.
- **Reduce:** Replace symbols on the stack with the left-hand side of a production.

Step-by-Step Parsing

Step	Stack	Remaining Input	Action
0	$	id + id $	Start
1	$ id	+ id $	Shift id
2	$ T	+ id $	Reduce T → id
3	$ E	+ id $	Reduce E → T
4	$ E +	id $	Shift +
5	$ E + id	$	Shift id
6	$ E + T	$	Reduce T → id
7	$ E	$	Reduce E → E + T
8	$ E	$	**Accept**

Explanation

Initially, the parser shifts the first id onto the stack:

$ id

Since id matches the right-hand side of T → id, it reduces:

$ id → $ T

Then T matches E → T, so another reduction gives:

$ T → $ E

The parser then shifts + and the second id:

$ E + id

The second id is reduced to T:

$ E + id → $ E + T

Finally, E + T matches the production:

E → E + T

so it is reduced to E:

$ E + T → $ E

Since the entire input has been consumed and the stack contains the start symbol E, the parser **accepts** the input.

Final Result

id + id

is successfully parsed according to the grammar, demonstrating how a **shift-reduce parser works from the bottom up**, gradually reducing the input symbols until the start symbol E is obtained.

5 Scenario: Operator Precedence Parsing

A parser encounters the expression a / b * c and must evaluate it using operator precedence parsing techniques. The parser relies on precedence relations such as yields precedence, equal precedence, and takes precedence to determine whether to shift the next symbol or reduce the current handle. Since division and multiplication have the same precedence and are left associative, the parser must correctly apply these rules to ensure proper evaluation order. In this scenario, the precedence table must be constructed and the parsing process should be traced to show how the expression is evaluated.

Scenario-Based Task: Operator Precedence Parsing

Given

a / b * c

1. Precedence Rules

Both / and * have the **same precedence** and are **left associative**. Therefore, the expression is evaluated from **left to right**:

(a / b) * c

2. Operator Precedence Table

Stack Symbol	/	*	a/b/c	$
/	>	>	<	>
*	>	>	<	>
a/b/c	>	>	—	>
$	<	<	<	Accept

- < = **Yields precedence** → Shift
- = = **Equal precedence**
- > = **Takes precedence** → Reduce

3. Parsing Trace

Condition
60 >= 60
True
False path/Skipped
result = "Try again"
True path/Runs
result = "Pass"
Continue:result = "Pass"
Matching code
if score >= 60:
result = "Pass"
else:
result = "Try again"
print(result)
score
score
Give feedback

Stack	Input	Action
$	a / b * c $	Shift a
$ a	/ b * c $	Shift /

$ a /	b * c $	Shift b
$ a / b	* c $	**Reduce** a / b
$ E	* c $	Shift *
$ E *	c $	Shift c
$ E * c	$	**Reduce** E * c
$ E	$	**Accept**

4. Final Evaluation

The parser first reduces:

a / b

Then it applies multiplication:

(a / b) * c

Final Answer

The expression a / b * c is evaluated as (a / b) * c because multiplication and division have equal precedence and are left associative. Operator precedence parsing shifts symbols when the incoming operator has higher precedence and reduces the handle when the stack operator takes precedence.

6 Scenario: SLR Parsing Table Construction

A compiler student is assigned the task of constructing an SLR parsing table for the grammar S → AA and A → aA | b. The process involves augmenting the grammar, generating the canonical collection of LR(0) items, computing FOLLOW sets, and building the parsing table with appropriate shift, reduce, and accept actions. Each state in the table represents a stage in the parsing process, and transitions between states are determined using grammar symbols. In this scenario, the student must describe the step-by-step procedure to construct the SLR parsing table and explain how it is used for efficient bottom-up parsing.

SLR Parsing Table Construction

Given grammar:

- **S → AA**
- **A → aA**
- **A → b**

1. Augment the Grammar

Add a new start symbol:

S′ → S

So the augmented grammar is:

1. S′ → S
2. S → AA
3. A → aA
4. A → b

---

2. Construct LR(0) Item Sets

I = Closure(S′ → ·S)

S′ → ·S
S → ·AA
A → ·aA
A → ·b

Transitions:

- GOTO(I , S) = I
- GOTO(I , A) = I
- GOTO(I , a) = I
- GOTO(I , b) = I

I

S′ → S·

I

S → A·A
A → ·aA
A → ·b

I

A → a·A
A → ·aA
A → ·b

I

A → b·

I

From GOTO(I , A):

S → AA·

I

From GOTO(I , A):

A → aA·

Additional transitions:

GOTO(I , a) = I
GOTO(I , b) = I
GOTO(I , a) = I
GOTO(I , b) = I

---

3. Compute FOLLOW Sets

For the grammar:

S → AA
A → aA | b

Since S is the start symbol:

FOLLOW(S) = { $ }

In S → AA, the first A is followed by another A, whose FIRST set contains a and b:

FOLLOW(A) = { a, b, $ }

Therefore:

FOLLOW(S) = { $ }
FOLLOW(A) = { a, b, $ }

---

4. Construct the SLR Parsing Table

Use:

- **shift** when the item contains a terminal after the dot.
- **reduce** when the dot is at the end of a production, using the FOLLOW set.
- **accept** for $S' \rightarrow S\cdot$.

Productions:

r1: $S \rightarrow AA$
r2: $A \rightarrow aA$
r3: $A \rightarrow b$

State	a	b	$	S	A
0	s3	s4	—	1	2
1	—	—	acc	—	—
2	s3	s4	—	—	5
3	s3	s4	—	—	6
4	r3	r3	r3	—	—
5	r1	r1	r1	—	—
6	r2	r2	r2	—	—

---

5. Parsing Procedure

The SLR parser uses a **stack**, the **input string**, and the parsing table.

For example, for:

aabb$

The parser repeatedly:

1. Looks at the current state on top of the stack.
2. Checks the next input symbol.
3. Uses the ACTION table.
4. Performs **shift**, **reduce**, or **accept**.
5. Uses the GOTO table after a reduction.
6. Continues until the input is accepted or an error occurs.

Conclusion

SLR parsing is an efficient **bottom-up parsing technique**. The canonical LR(0) items identify the parser states, while the FOLLOW sets determine where reductions are allowed. For this grammar, the constructed SLR table has no conflicts, so the grammar can be parsed successfully using shift-reduce operations.

RUBRICS FOR CALCULATION:

Scenario based assessment

Criteria	Weigh tage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand